

Experiment No 07

Aim: Introduction to Truffle, Establish local blockchain using Truffle and deployment of Smart Contract

Theory

1. Blockchain-Based Rental Deposit Protection System

A Blockchain-Based Rental Deposit Protection System is a decentralized application in which a tenant's rental security deposit is managed using a smart contract. Instead of directly giving the security deposit to the landlord, the tenant deposits the amount into a smart contract.

The smart contract temporarily holds the deposit and releases it according to predefined conditions. This provides a transparent and secure way of managing rental deposits because all transactions are recorded on the blockchain.

In this mini-project, two users are considered:

- **Landlord** – deploys the smart contract and can release the deposit.
- **Tenant** – pays the rental security deposit into the smart contract.

The basic working is:

Tenant → Pays Deposit → Smart Contract Holds Deposit → Landlord Releases Deposit → Deposit Returned to Tenant

2. Role of Blockchain in the System

Blockchain is a distributed digital ledger that stores transactions in a secure and transparent manner. Once a transaction is recorded on the blockchain, it becomes part of the blockchain transaction history.

In the Rental Deposit Protection System, blockchain is used to record important activities such as:

- Deployment of the rental deposit smart contract.
- Payment of the security deposit by the tenant.
- Storage of the deposit in the smart contract.
- Release of the deposit by the landlord.
- Transfer of the deposit back to the tenant.

This makes the deposit process more transparent because blockchain transactions can be verified.

3. Smart Contract

A smart contract is a program stored and executed on a blockchain. It automatically performs operations according to the rules written in its code.

In this mini-project, the RentalDepositProtection smart contract manages the security deposit between the landlord and tenant.

The smart contract stores information such as:

- Landlord's blockchain address.
- Tenant's blockchain address.
- Deposit amount.
- Whether the deposit has been paid.
- Whether the deposit has been released.

The contract also contains functions for paying and releasing the rental deposit.

4. Landlord and Tenant Accounts

Two blockchain accounts are used in the system.

Landlord Account: The landlord account is used to deploy the smart contract. During deployment, the address of the account deploying the contract is stored as the landlord's address. Only this landlord is allowed to release the security deposit.

Tenant Account: The tenant account represents the person renting the property. The tenant's public wallet address is provided while deploying the smart contract. Only the registered tenant is allowed to pay the rental deposit.

Thus, blockchain addresses are used to identify and distinguish the landlord and tenant.

5. Ganache Local Blockchain

Ganache is a local Ethereum blockchain used for developing and testing blockchain applications. It allows blockchain transactions to be performed without using real cryptocurrency.

Ganache provides multiple test accounts containing fake ETH. In this mini-project, Ganache accounts are used as the Landlord and Tenant accounts.

Ganache is useful because it provides:

- Pre-funded Ethereum accounts.
- Fake ETH for testing.
- Fast transaction processing.
- Local blockchain environment.
- Transaction history.
- Contract creation details.
- Gas usage and transaction hashes.

Therefore, the complete rental deposit process can be tested without spending real money.

6. MetaMask

MetaMask is a cryptocurrency wallet that allows users to manage blockchain accounts and interact with Ethereum-compatible blockchain networks.

In this mini-project, Ganache accounts are imported into MetaMask using their test private keys. The accounts are identified as the **Landlord** and **Tenant**.

MetaMask can also be connected to the Ganache Local network using its RPC URL. This allows the user to view and manage the test ETH associated with the Ganache accounts.

Note: Private keys should never be shared for real cryptocurrency accounts. Ganache private keys are used here only because the accounts are local test accounts.

7. Remix IDE

Remix IDE is a browser-based development environment used for creating, compiling, deploying and interacting with Solidity smart contracts.

In this project, Remix is used to:

- Write the RentalDepositProtection smart contract.
- Compile the Solidity program.
- Connect to the Ganache blockchain.
- Deploy the smart contract.
- Pay the rental deposit.
- Release the rental deposit.
- Check the values stored in the contract.

Thus, Remix provides the interface between the Solidity program and the local Ethereum blockchain.

8. Solidity

Solidity is a programming language used for developing smart contracts for Ethereum and other Ethereum-compatible blockchain networks.

The Rental Deposit Protection System is implemented using Solidity. The Solidity contract defines the rules regarding who can pay the deposit, who can release it, and where the deposited ETH should be transferred.

For example, conditions are used to ensure that:

- Only the tenant can pay the deposit.
- The deposit cannot be paid more than once.
- Only the landlord can release the deposit.
- A deposit must exist before it can be released.
- The deposit cannot be released more than once.

Therefore, Solidity is used to implement the business logic of the rental deposit system.

9. Smart Contract Deployment

Smart contract deployment is the process of placing a compiled smart contract onto a blockchain.

In this mini-project, the Landlord account deploys the RentalDepositProtection contract. During deployment, the Tenant's public blockchain address is provided to the constructor.

The contract then stores:

Landlord Address = Address of the account deploying the contract

Tenant Address = Address supplied during deployment

Once deployed, the smart contract receives its own unique contract address on the Ganache blockchain.

10. Rental Deposit Payment

The rental security deposit is paid using the payDeposit() function of the smart contract.

Only the registered Tenant account is allowed to call this function. When the tenant sends ETH with the transaction, the amount is transferred to the smart contract.

The smart contract then records:

- The amount deposited.
- That the deposit has been successfully paid.

The deposited ETH remains in the smart contract instead of being directly transferred to the landlord.

Thus:

Tenant → Rental Deposit → Smart Contract

This allows the smart contract to act as an intermediary for protecting the deposit.

11. Rental Deposit Release

The rental deposit can be released using the `releaseDeposit()` function.

Only the Landlord account is allowed to execute this function. Before releasing the deposit, the smart contract verifies that:

1. The caller is the landlord.
2. The tenant has already paid the deposit.
3. The deposit has not already been released.

After these conditions are satisfied, the smart contract transfers the stored deposit back to the Tenant account.

Thus:

Smart Contract → Released Deposit → Tenant

The contract also updates the deposit status to indicate that the amount has been released.

12. Gas Fee

A gas fee represents the computational cost required to execute a transaction or smart contract operation on an Ethereum-compatible blockchain.

Gas is required for operations such as:

- Deploying the smart contract.
- Paying the rental deposit.
- Releasing the rental deposit.

Since Ganache is being used in this experiment, the gas cost is paid using **fake test ETH** provided by Ganache. Therefore, no real cryptocurrency or money is required.

Your original experiment also introduces gas as the cost of processing blockchain transactions.

Blockchain_Experiment4

13. Transaction Hash

A transaction hash is a unique identifier generated for a blockchain transaction.

Transactions such as smart contract deployment, deposit payment and deposit release receive their own transaction hashes.

The transaction hash can be used to identify and verify a particular transaction.

For example:

Transaction Hash → 0x...

In Ganache, the transaction hash can be viewed along with other transaction information. This follows the same concept used in your existing experiment, where the transaction hash is used as a unique value for locating and verifying a transaction. Blockchain_Experiment4

14. Transaction Verification Using Ganache

Ganache provides a transaction history that can be used to verify the operations performed during the experiment.

For each transaction, information such as the following can be checked:

- Transaction hash.
- Sender address.
- Contract address.
- Amount transferred.
- Gas used.
- Block information.
- Contract creation.

Therefore, Ganache acts as the local blockchain transaction viewer for this experiment.

15. Working of the Rental Deposit Protection System

The overall working of the system is:

Step 1: Ganache creates the local Ethereum blockchain and provides test accounts.

Step 2: Two Ganache accounts are used as the **Landlord** and **Tenant**.

Step 3: The Ganache Local network is configured for interaction with the blockchain.

Step 4: The RentalDepositProtection smart contract is written and compiled using Remix IDE.

Step 5: The Landlord deploys the smart contract by providing the Tenant's public address.

Step 6: The deployed contract stores the Landlord and Tenant addresses.

Step 7: The Tenant calls payDeposit() and transfers the rental security deposit to the smart contract.

Step 8: The smart contract securely holds the deposited test ETH.

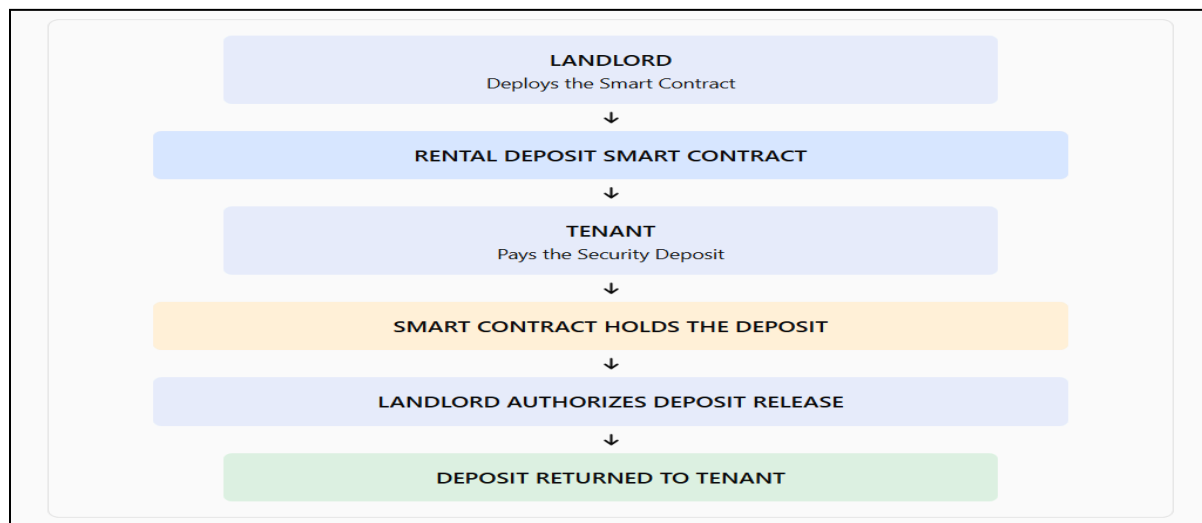
Step 9: The deposit amount and payment status can be verified from the contract.

Step 10: The Landlord calls releaseDeposit() after completion of the rental process.

Step 11: The smart contract transfers the security deposit back to the Tenant.

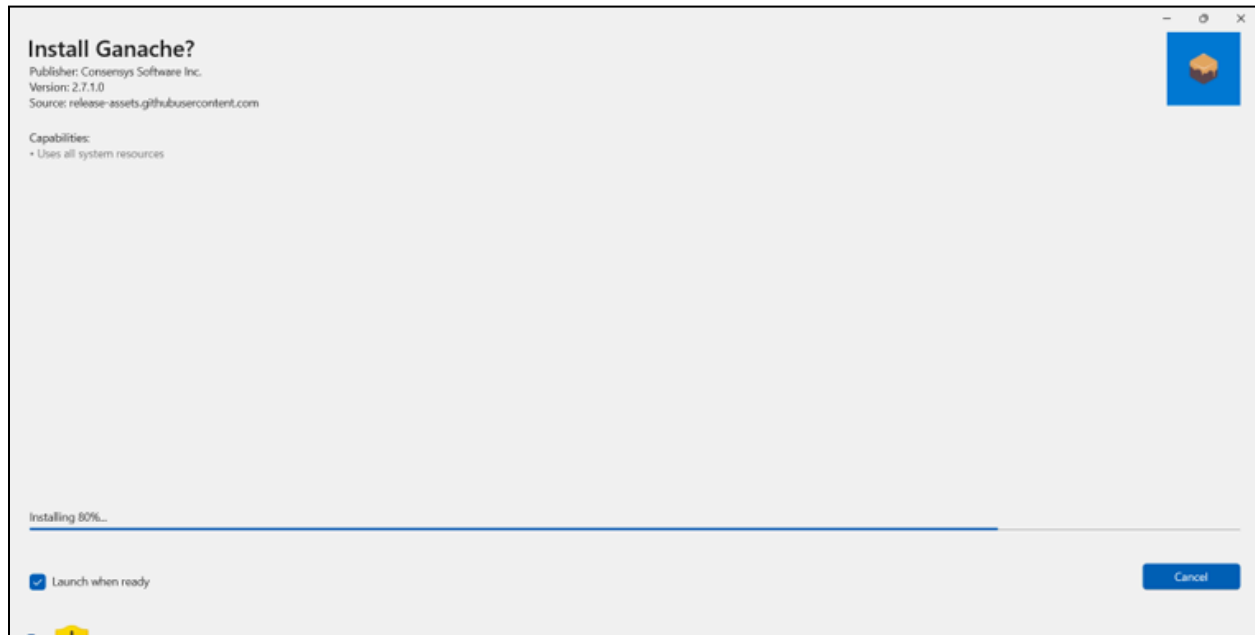
Step 12: All blockchain transactions are verified using Ganache.

Overall flow :

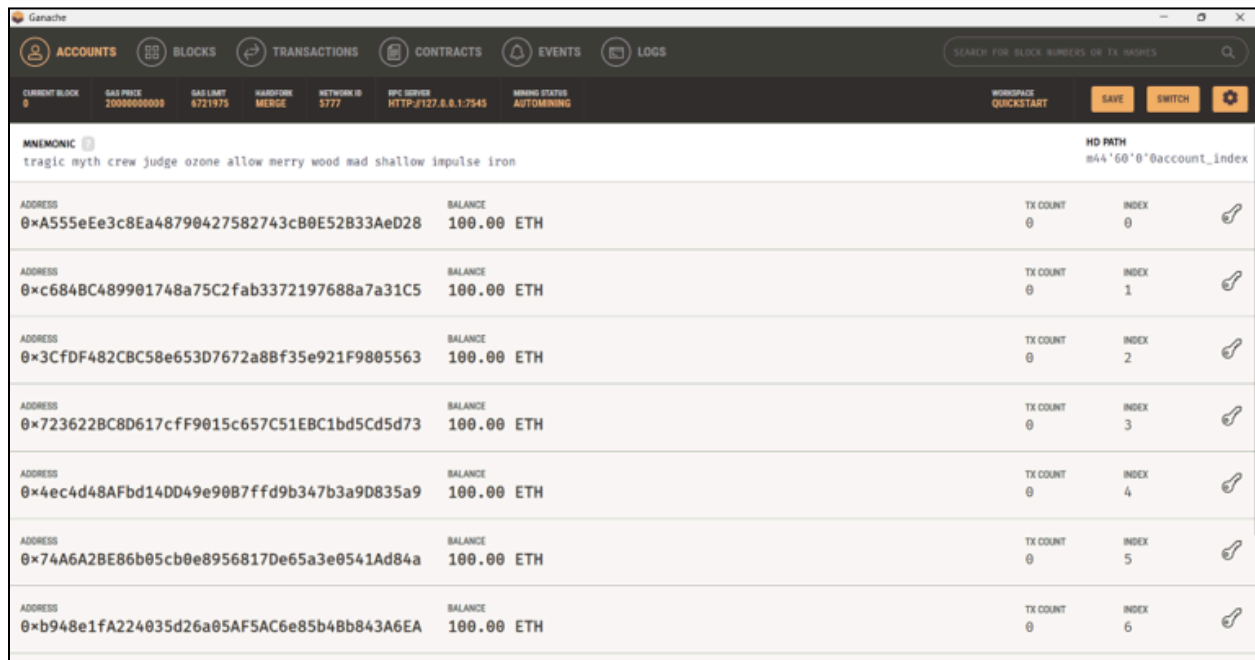


OUTPUT:

Ganache Installation



Ganache interface



RPCURL

WORKSPACE

SERVER

ACCOUNTS & KEYS

CHAIN

ADVANCED

ABOUT

CANCEL

RESTART

⚠ Restarting the Quickstart workspace resets the blockchain. All transactions and contract states will be reset.

SERVER

HOSTNAME

127.0.0.1 - Loopback Pseudo-Interface 1

The server will accept RPC connections on the following host and port.

PORT NUMBER

7545

NETWORK ID

5777

Internal blockchain identifier of Ganache server.

AUTOMINE

☒

Process transactions instantaneously.

ERROR ON TRANSACTION FAILURE

☒

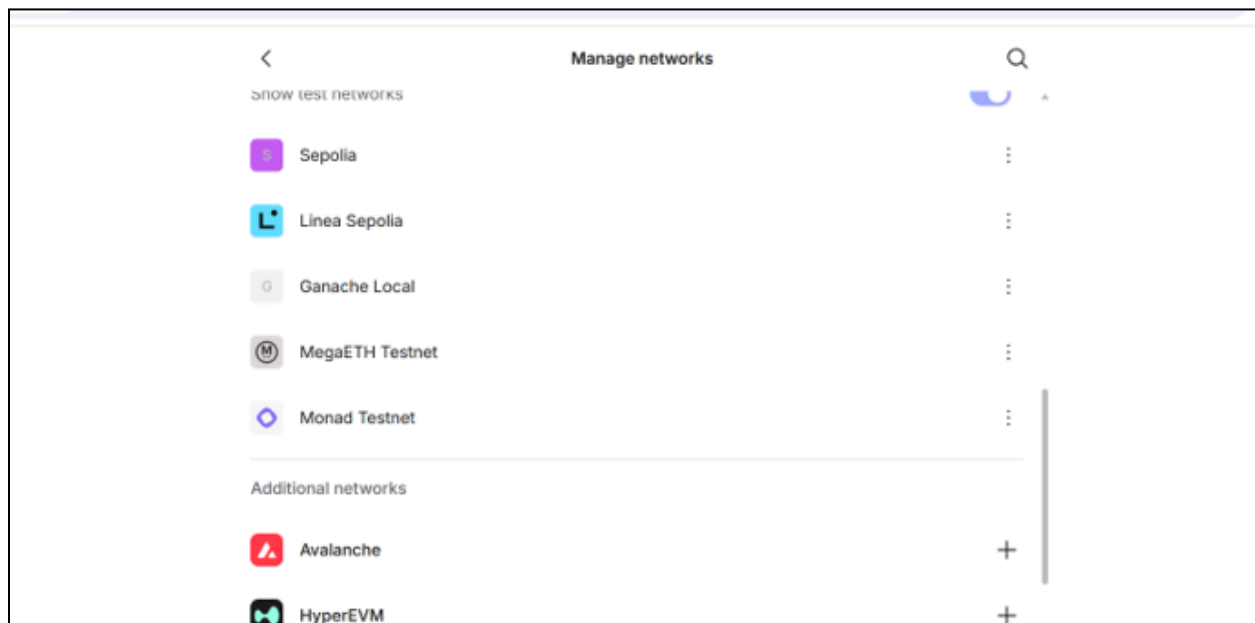
When transactions fail, throw an error. If disabled, transaction failures will only be detectable via the "status" flag in the transaction receipt. Disabling this feature will make Ganache handle transaction failures like other Ethereum clients.

CHAIN FORKING

☐

Fork an existing chain creating a new sandbox with the existing chain's accounts, contracts, transactions and data.

New network added in metamask



New account Imported(Landlord)

Imported Account 1

0xA555e...AeD28

\$0.00

+\$0.00 (+0.00%) [Discover](#)

\$ Buy

↕ Swap

↗ Send

↙ Receive

NEAR +100% over 30 days

Token for AI native chain hovers near \$5

Tokens

Perps

DeFi

NFTs

Activity

Network: Arc

USDC

-0.00%

\$0.00

0 USDC

Account imported

New Account imported (Tenant)

Tenant

0xc684B...a31C5

100 ETH

+\$0.00 (+0.00%) [Discover](#)

\$ Buy

↕ Swap

↗ Send

↙ Receive

NEAR +100% over 30 days

Token for AI native chain hovers near \$5

Tokens

Perps

DeFi

NFTs

Activity

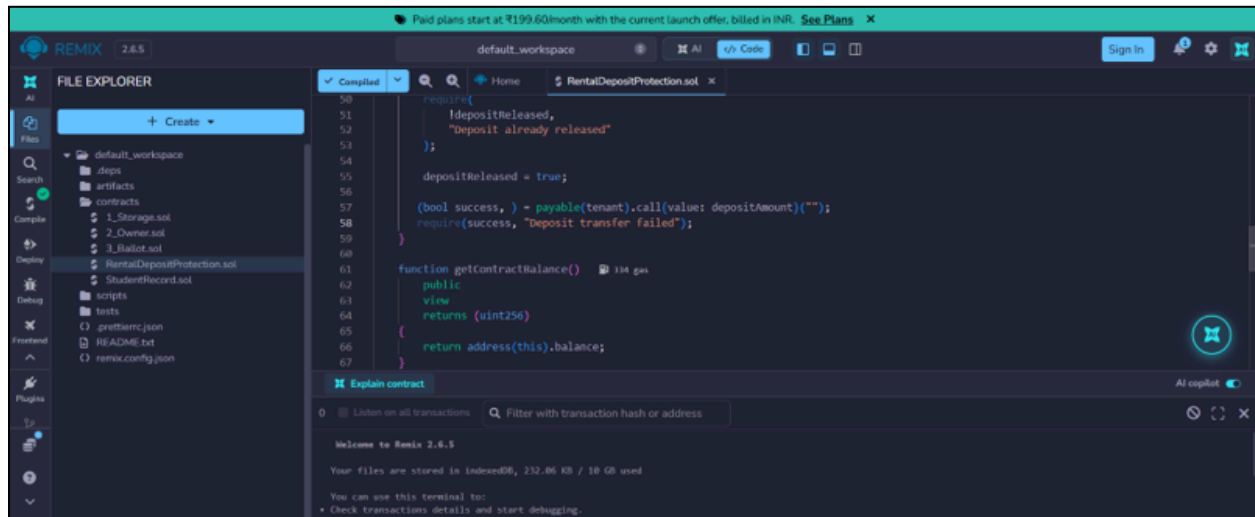
Network: Ganache Local

Ethereum

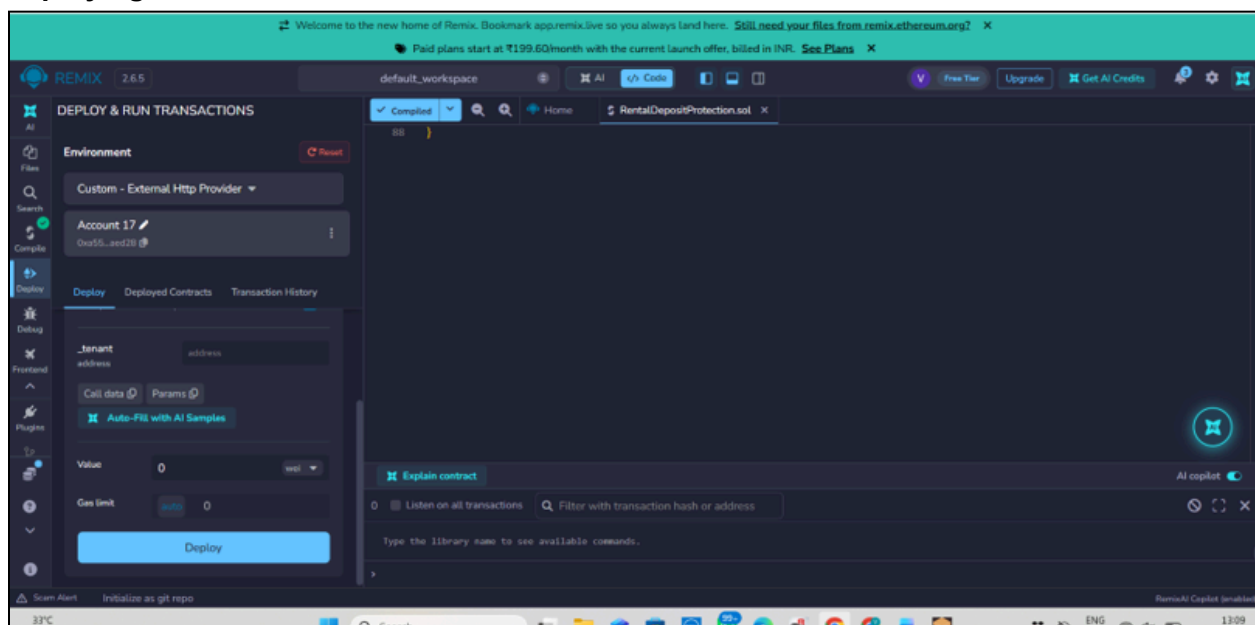
-

100 ETH

Creating a solidity contract and Compiling



Deploying the contract



Transaction processing

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is visible, showing the 'Environment' section with 'Account 17' and a balance of 99.996 ETH. The 'Deploy' section shows the contract name 'RentalDepositProtection.sol' and the 'tenant' address '0xc684BC489901748a75C2i'. The 'Value' is set to 0 and the 'Gas limit' is set to 0. The 'Deploy' button is highlighted. The main editor area shows the 'RentalDepositProtection.sol' file with the following code:

```
88 }
```

The 'Explain contract' panel on the right shows the following libraries are accessible:

- ethers.js

The panel also displays error messages:

```
error encoding arguments: TypeError: invalid address (argument="address", value="", code=INVALID_ARGUMENT, version=6.14.6) (argument="", value="", code=INVALID_ARGUMENT, version=6.14.6)
```

The execution might have thrown on the EVM version used by the selected environment is not compatible with the compiler EVM version.

Contract deployed

The screenshot shows the Remix IDE interface after the successful deployment of the 'RentalDepositProtection.sol' contract. The 'DEPLOY & RUN TRANSACTIONS' sidebar is visible, showing the 'Environment' section with 'Account 17' and a balance of 99.995 ETH. The 'Deploy' section shows the contract name 'RentalDepositProtection.sol' and the 'tenant' address '0xc684BC489901748a75C2i'. The 'Value' is set to 0 and the 'Gas limit' is set to 0. The 'Deploy' button is highlighted. The main editor area shows the 'RentalDepositProtection.sol' file with the following code:

```
10 bool public depositPaid;
11 bool public depositReleased;
12
13 constructor(address _tenant) {
14     landlord = msg.sender;
15 }
```

The 'Explain contract' panel on the right shows the following libraries are accessible:

- ethers.js

The panel also displays error messages:

```
creation of RentalDepositProtection errored: error occurred: Transaction canceled by user..
Transaction canceled by user..
If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

The transaction was successful, as indicated by the green checkmark and the message:

```
[block:2 txindex:0] From: 0xd35...a028 to: RentalDepositProtection.(constructor) value: 0 wei data: 0x60...a11c3 logs: 0 hash: 0xc21...fb3e
```

Deposit Paid by the tenant

REMIX 2.6.5

default_workspace

AI `</> Code`   

Free Tier

Upgrade

AI

Files

Search

Compile

Deploy

Debug

Frontend

Plugins

⋮

⋮

⋮

⋮

DEPLOY & RUN TRANSACTIONS

Environment

Custom - External Http Provider

Account 18  97.999 ETH

Deploy

Deployed Contracts

Transaction History

landlord Call

payDeposit Transact

releaseDeposit Transact

tenant Call

Low level interaction +

Value ether

0 ☐ Listen on all transactions

Filter with transaction hash or address

status 1 Transaction mined and execution completed

transaction hash 0x9f92c79bc2b83aad9662d3801842e8a9588c8b8e7883d19a082afa1a45ee78c 

block hash 0x372d53e497bedeff9b7d85e64b22cd23c40fa2418c9965767c9e08f11cfc1e34 

block number 5 

from 0xc6048c4899e174ba75c2fab372197688a7a31c5 

to RentalDepositProtection.payDeposit() 0xbA19C2897F326638885802552e430674027E7836 

transaction cost 67857 gas 

decoded input {} 

decoded output - 

logs [] 

raw logs [] 

value 2000000000000000000 wei 

Verifying Deposit paid

raw logs	[]
value	200000000000000000 wei
CALL [call] from: 0xc684bc489901748a75c2fab3372197688a7a31c5 to: RentalDepositProtection.depositPaid() data: 0x	
from	0xc684BC489901748a75C2fab3372197688a7a31C5
to	RentalDepositProtection.depositPaid() 0xbA19C2897F32663B8850D2552e430A74027E7036
input	0xef3...ab330
output	00000000000000000000000000000001
decoded input	{ }
decoded output	{ "0": "bool: true" }
logs	[]
raw logs	[]

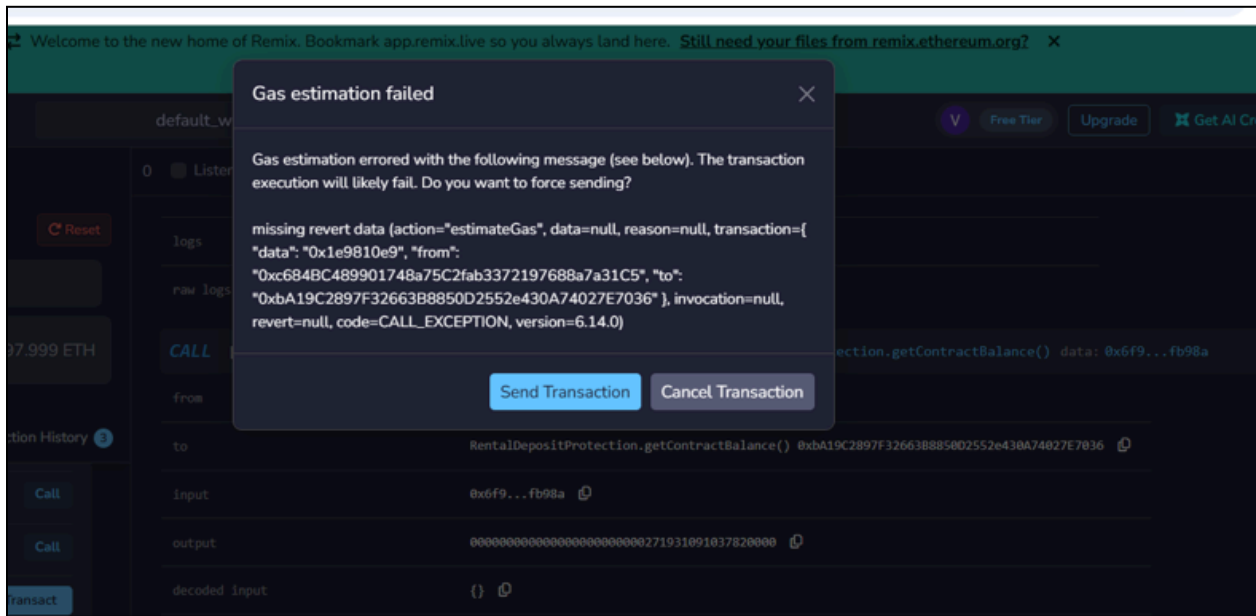
Deposit amount

CALL [call] from: 0xc684bc489901748a75c2fab3372197688a7a31c5 to: RentalDepositProtection.depositAmount() data: 0x419...759f5	
from	0xc684BC489901748a75C2fab3372197688a7a31C5
to	RentalDepositProtection.depositAmount() 0xbA19C2897F3266388850D2552e430A74027E7036
input	0x419...759f5
output	000000000000000000000000271931091037820000
decoded input	{}
decoded output	{ "0": "uint256: 200000000000000000" }
logs	[]
raw logs	[]

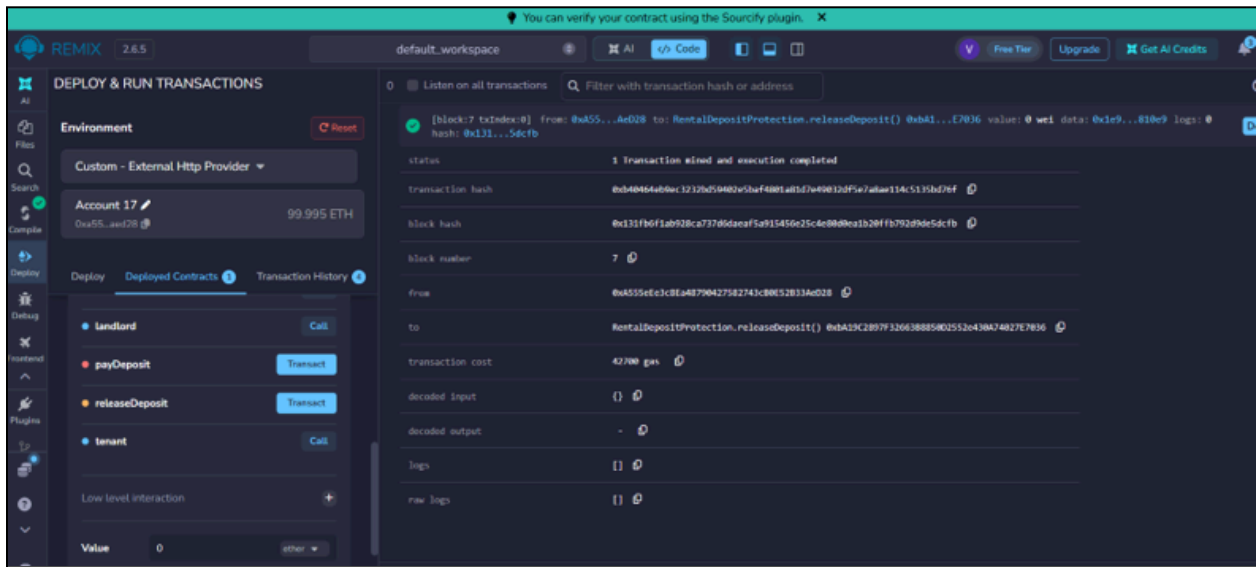
Contract balance

Custom - External Http Provider	
Account 18 97.999 ETH	
0xc68...a31c5	
Deploy Deployed Contracts Transaction History	
depositPaid Call	
depositReleased Call	
getContractBalance Call	
uint256: 20000000000000000000	
getRentalDetails Call	
landlord Call	
payDeposit Transact	
releaseDeposit Transact	
raw logs []	
CALL [call] from: 0xc684bc489901748a75c2fab3372197688a7a31c5 to: RentalDepositProtection.getContractBalance() data: 0x6f9...fb98a	
from	0xc684BC489901748a75C2fab3372197688a7a31C5
to	RentalDepositProtection.getContractBalance() 0xbA19C2897F3266388850D2552e430A74027E7036
input	0x6f9...fb98a
output	000000000000000000000000271931091037820000
decoded input	{}
decoded output	{ "0": "uint256: 200000000000000000" }
logs	[]
raw logs	[]

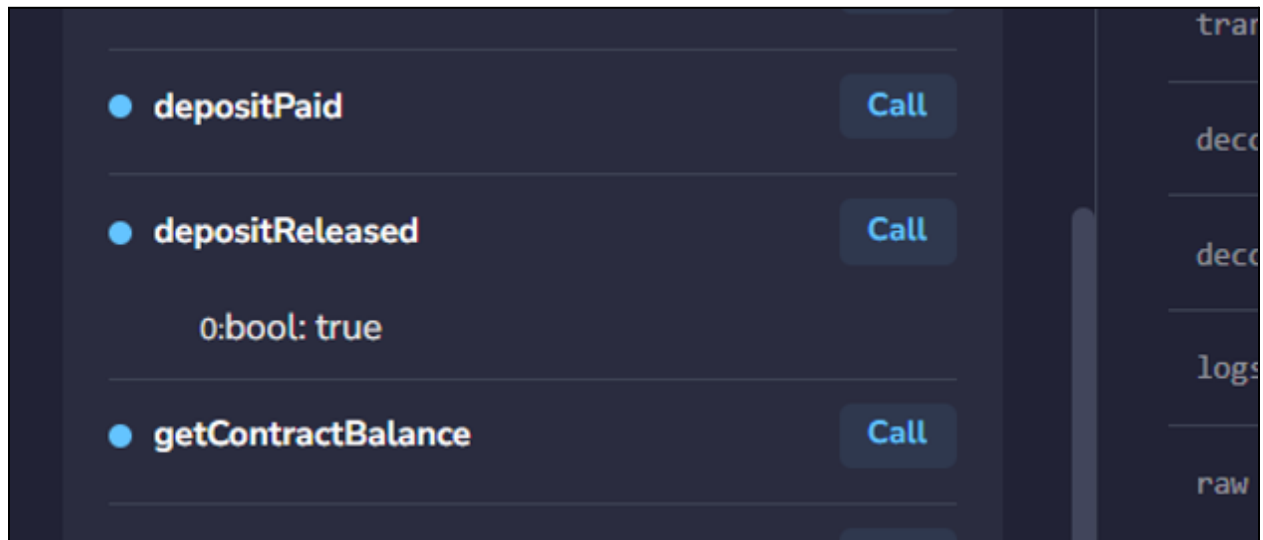
Calling release deposit gives error as only landlord can release the deposit



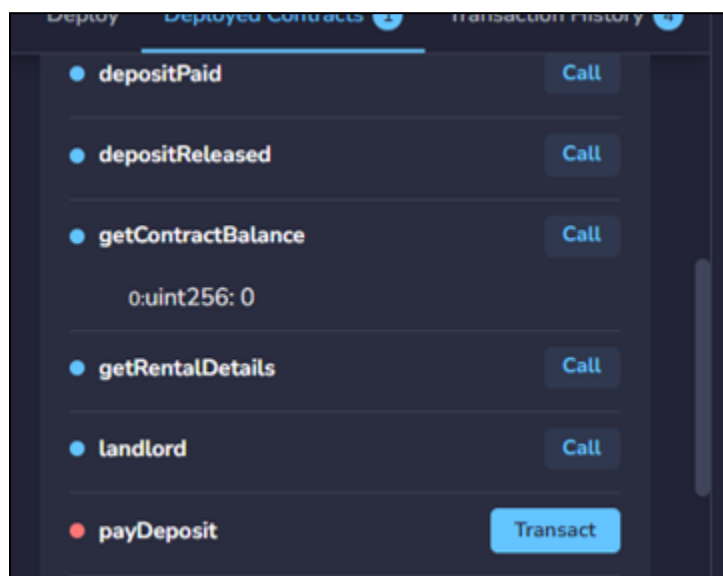
Deposit released by landlord



Verifying whether deposit was released



Contract balance after release



Ganache Interface showing all the transactions

TX HASH	FROM ADDRESS	TO CONTRACT ADDRESS	GAS USED	VALUE	
0xb40464eb9ec3232bd59402e5baf4801a81d7e49032df5e7a8ae114c5135bd76f	0xA555eE3c8Ea48790427582743c80E52833AeD28	0xbA19C2897F326638856D2552e438A74027E7036	42780	0	CONTRACT CALL
0x277208c1a410354eb5221a5a631795352d0f434d28d984f345f5034f6c4e86ac	0xc6848C489901748a75C2fab3372197688a7a31C5	0xbA19C2897F326638856D2552e438A74027E7036	21188	2000000000000000000	CONTRACT CALL
0x9f92c79bc2b03aad9662d3891042e0a9588c6b8e7803d19a982a6a1a45eec70c	0xc6848C489901748a75C2fab3372197688a7a31C5	0xbA19C2897F326638856D2552e438A74027E7036	67857	2000000000000000000	CONTRACT CALL
0x3094532fc8afaed65e1426898ee8b0f56fd0b49fff3dea3581427e56b94fb16	0xA555eE3c8Ea48790427582743c80E52833AeD28	0xbA19C2897F326638856D2552e438A74027E7036	23688	2000000000000000000	CONTRACT CALL
0x11468551eb205d61fd842e8df171a0ce3c3f013796ccb0541c1306ddad153726		CREATED CONTRACT ADDRESS			CONTRACT CREATION

Conclusion:

Thus, a Blockchain-Based Rental Deposit Protection System was successfully implemented using Solidity, Remix IDE, MetaMask and Ganache. Two blockchain accounts were used to represent the Landlord and Tenant. The smart contract was deployed on the Ganache local blockchain, and the Tenant's rental security deposit was stored in the smart contract. The deposit was later released by the Landlord and transferred back to the Tenant. The corresponding contract deployment and transactions were verified using Ganache. This experiment demonstrated how blockchain and smart contracts can be used to provide a transparent and controlled mechanism for managing rental security deposits.